



ÉCOLE POLYTECHNIQUE
FÉDÉRALE DE LAUSANNE

Global Control Logic Design and Integration on SpeedGoat

Semester Project

Author
KOBLER TIM

Professor
HODDER ANDRÉ

Supervisor
AMACHER ROBIN, BERVILLÉ JULES, MROUEH MARWANE



Swiss Solar Boat

Spring 2026

Contents

1	Abstract	3
2	Introduction	4
2.1	Global Context	4
2.2	Project's Context	5
3	Project Structure	6
3.1	Folder Hierarchy	6
3.2	Key Files	6
3.3	Boat_main.slx Structure	7
3.3.1	Main FSM	7
3.3.2	Subsystems	8
3.3.3	I/O Layer	8
3.3.4	Data Logging	9
4	Main FSM	10
4.1	Truth Tables	10
4.2	Safety FSM	10
4.2.1	Init	12
4.2.2	Startup	12
4.2.3	Functional	12
4.2.4	Safety Shutdown	13
4.2.5	Emergency Shutdown	13
4.2.6	Maintenance	14
4.3	User FSM	15
5	Subsystems	16
5.1	Low Voltage Battery - LVBAT	16
5.2	Solar Panels - PV	16
5.3	Dashboard - DASH	17
5.3.1	Emergency and Fault Detection	17
5.3.2	Normal Switch FSM	17
5.3.3	Selector FSM	17
5.3.4	Switch+LED FSM	18
5.3.5	Solenoid Switch FSM	18
5.4	High Voltage - HV	19
5.5	Energy Management System - EMS	19
5.6	Hydrogen - H2	20
5.7	Cooling	20
5.8	CPU and Overload Monitoring - Other	21
5.9	Retraction	21
5.10	Control	22
5.10.1	Throttle	23
5.10.2	Steering Control	23
5.10.3	Gantry synchroniser	23
5.10.4	Subtleties	23
6	Desing Choices	24
6.1	Code Modularity	24
6.2	Frequencies	24
6.3	XCP	25
6.3.1	A2L File Generation	25
6.4	Simulink Specifics	25

7	Tests	26
7.1	Main FSM Testing	26
7.2	Subsystem Testing	26
7.3	System As-a-whole Testing	26
7.4	Logs Analysis	27
8	Summary of the results	28
8.1	Solutions and important informations	28
8.2	Workflow	28
9	Challenges	30
9.1	Finite State Machine (FSM) Testing	30
9.2	Compilation and Simulation Errors	30
9.3	DBC Managment	30
9.4	Teamwork and Simulink Merges	30
10	Conclusion	31
10.1	Next Steps	31

1 Abstract

This report presents the design and integration of the global low-voltage control logic for the Lobst'air, a three-passenger hydrogen-powered foiling boat developed by Swiss Solar Boat at EPFL. The software, implemented in MATLAB Simulink and executed in real-time on an onboard Speedgoat computer, unifies all subsystems under a modular, safety-first architecture centered around a hierarchical finite state machine. The work was carried out during the final assembly and commissioning semester, with the primary goal of delivering a fully operational boat.

Abbreviations and terminology

ATEX - European directives for explosive atmospheres
CAN - Controller Area Network
DBC - Matlab CAN database file format
EMS - Electrical Managment System
ESC - Electronic Speed Controller (for the torpedoes)
FSM - Finite State Machine
Gantry - Main foil of the Lobst'air
GPIO - General-Purpose input/output
GS - Ground Station (user interface available on internet)
HV - High Voltage subsystem
HVIL - High-Voltage Interlock Loop
H2 - Hydrogen subsystem
I/O - Input/Output
LEMO - Push-Pull connector manufacturer
LV - Low Voltage subsystem
MCU - Microcontroller
NTC - Temperature sensor
OTV - On Tank Valve of the H2 tank
PWM - Pulse-Width Modulation
Raspi - Raspberry Pi
RPM - Revolution per minute
Speedgoat - Real time on board computer used on the boat
SRT - Simulink Real Time
Struts - Vertical foils of the gantry system
XCP - Universal Measurement and Calibration Protocol

2 Introduction

2.1 Global Context

This part is common to every SSB report; please skip it if you know what the global context is.

After three participations at the MEBC in 2021, 2022 and 2025 with the Dahu, the Swiss Solar Boat team is now turning to a new challenge aiming at placing itself halfway between pure optimization for the purpose of a competition and emerging industrial actors in the field of sustainable boating. To do so, the team has decided to densify the energy on board and reach higher speeds than the previous boat. This is achieved by using pressurized hydrogen gas as an energy carrier.

In this state of mind Swiss Solar Boat developed a new boat for 2026 with the following main characteristics:

- 3 passengers
- 160 km of autonomy
- 25 kts cruising speed
- 35 kts of top speed
- Electric propulsion powered by a fuel cell and PVs

It is in this context that this report is made. It aims to describe and present the work and results obtained in the framework of a semester project on the design and production of this new boat. The project took place during the 6th semester of the project of this new boat (Figure 13) the one of the launch.



Figure 1: 3D rendering of the Lobst'air

2.2 Project's Context

This project was conducted during the final assembly phase of the Lobst'air boat. It establishes the core software foundation required for initial integrated system testing. While individual electronic subsystems were developed and validated in previous semesters, they were finalized in parallel with this main software architecture, which unifies all pre-existing codebases. The primary focus of this project is the structure of the main control code and the integration of all components to transition from a testbench environment to the fully assembled boat.

Efficient integration required close collaboration with three other credited projects. The reports of Romain Mauris, Alessandro Lupi, Margaux Lari and Beltran Albert are available at the following link. Alignment was achieved through frequent joint working sessions and continuous inter-team communication.

In alignment with the overall project objective of competing in Monaco, the primary success metric was delivering a functional boat before the end of the semester. Project progress was driven by success, delays, and intensive debugging sessions to ensure operational readiness while simultaneously establishing the foundation for future optimization.

This report details the developed Simulink and MATLAB architecture. This software executes in real-time on the boat's onboard computer, the Speedgoat.



Figure 2: Picture of the second time the Lobstaire reached 14 kn

3 Project Structure

The project structure provides a clean environment to easily manage, understand, and maintain all source files.

All files are hosted on the SSB GitLab instance in the Boat repository [1]. While the repository contains multiple branches, the `main` branch is always up to date and working. Other branches are dedicated to development, code merging, and testing; these follow explicit naming conventions typically corresponding to a specific subsystem development or test phase. All the code was created in MATLAB Simulink. Please refer to the Simulink documentation [2] for more technical information.

Important

This codebase is under continuous development. While this report provides an overview, always refer directly to the source code for the most current details, as descriptions or figures herein may be slightly outdated.

3.1 Folder Hierarchy

All important files are in the main directory. Subdirectories are structured as follows to maintain project organization:

- **Helpers:** Contains all the Matlab files (`.m`) that contains constants or variables used in Simulink
- **boat_main_sg_rtw :** Contains the generated `C++` compiled files deployed directly to the Speedgoat for real-time execution.
- **dbc:** Contains `.dbc` network definition files and associated libraries for CAN bus communication.
- **data_types:** Contains Simulink bus definitions.
- **data_types/enum:** Contains MATLAB scripts (`.m`) defining enumerations utilized within Simulink.
- **crashdumptemp:** Temporary directory for utility and crash dump files generated by Simulink.

3.2 Key Files

- **boat_logic.prj:** The primary Simulink project file that manages file dependencies. Opening this project ensures all required paths and references are correctly resolved.
- **init_project.m:** Script defining the project links and environment setup. This script must be re-executed if any `.m` file within the `helpers` directory is modified.
- **boat_main.slx:** The main Simulink model. This file is compiled and deployed directly to the Speedgoat target.
- **library.slx:** A shared Simulink library containing reusable blocks. Updating a block within this library propagates the changes across all model instances.
- **helpers/frequency_cte.m:** Configuration script defining all execution frequencies used throughout the Simulink model.
- **helpers/XCP_variables.m:** Defines the parameter variables accessible and modifiable via the XCP protocol during real-time execution on the Speedgoat.

Remaining files in the directory are required for operation but typically do not require user modification.

3.3 Boat_main.slx Structure

The main Simulink model is partitioned into distinct subsystems to facilitate efficient debugging and system isolation. This modular architecture separates the core control logic from the individual hardware segments of the boat, as illustrated in Figure 3. It is recommended to view the Simulink model alongside this documentation for optimal visual context.

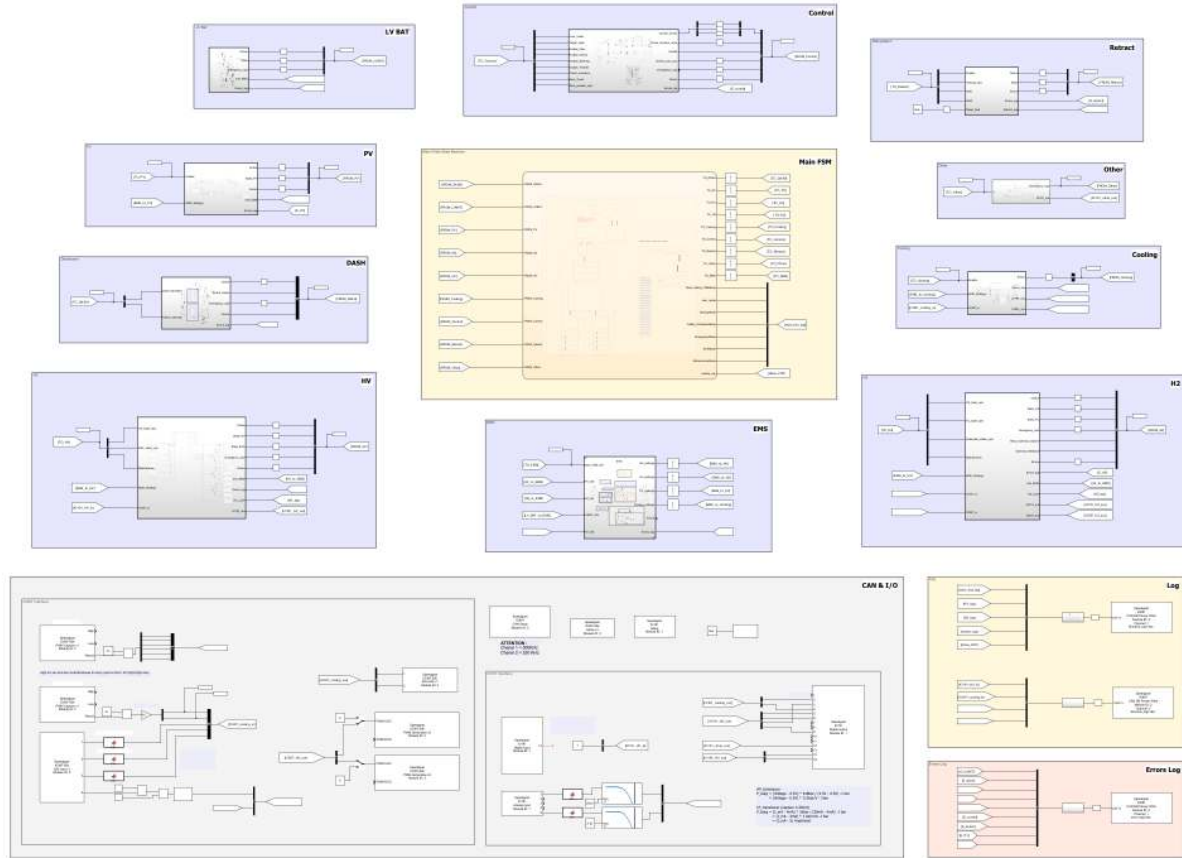


Figure 3: Boat_main.slx file. This file is the main code that runs on the boat. This image is here to have a global overview of the code.

3.3.1 Main FSM

The main Finite State Machine (FSM), located in the yellow block, serves as the primary controller for the software architecture. It gets status inputs from all subsystems and outputs high-level operational commands to change their behavior. This core FSM is safety-driven and explicitly manages all emergency procedures. A detailed breakdown of the main FSM architecture is provided in Section 4.

3.3.2 Subsystems

Each distinct subsystem is contained within an individual blue block. The architecture utilizes both standard Simulink subsystems and referenced subsystems (independent files imported into the main model). Model referencing is implemented to enable concurrent development by multiple engineers while preventing merge conflicts within the primary Simulink file. Detailed guidelines on collaborative workflows and model merging are covered in Section 9.4.

The specific subsystems are detailed in Section 5, and their standardized internal architecture is illustrated in Figure 4.

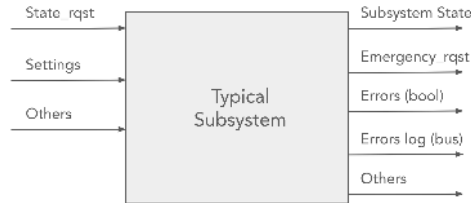


Figure 4: Typical subsystem interface. All inputs are on the left and outputs on the right. "rqst" means "request."

Subsystems can independently initiate an emergency sequence by raising an emergency request flag, a process detailed in Section 7. If an internal fault occurs within a subsystem, its local error boolean is driven high, and a dedicated error logging bus propagates the specific fault code to the main data logging system. Additional inputs and outputs are implemented across individual subsystems for specific interfacing requirements.

3.3.3 I/O Layer

The gray block represents the I/O layer, which interfaces all non-CAN sensors directly with the Speedgoat hardware. The SRT blocks are configured using three primary, centralized blocks that are unique across the entire project architecture.

- Speedgoat IO691 CAN setup
- Speedgoat IO397-50k setup v3
- Speedgoat IO191 setup

It is important to mention that all the SRT blocs can only be used in the `boat_main` file. IO191 and IO397 blocks can be used only once in the same file.

- **IO191 Module:** Interfaces with all digital GPIO pins. It controls binary actuators such as valves, pumps, and status LEDs, and reads binary sensors including the HVIL and the high/low pressure switches.
- **IO397 Module:** Interfaces with all analog channels. This module processes analog sensor signals, flowmeter inputs, cooling fan tachometer frequencies, and generates PWM control outputs.
- **CAN DB Parser Blocks:** Instantiated directly within the top-level `boat_main.slx` model rather than inside referenced subsystems. This centralized routing maintains clean subsystem interfaces and simplifies signal tracing.

All the I/O connections are visible in the main Simulink. Refer to `boat_main.slx` for specific pinout configurations.

3.3.4 Data Logging

Data logging was done in 3 different ways.

- **CAN data logging:** Transmits all FSM states and analog sensor values over the CAN bus to the raspi for real-time monitoring on the ground station screens.
- **CAN error logging:** Aggregates all internal fault signals generated dynamically within the sub-systems and transmits them via CAN to the raspi for ground station visualization.
- **Simulink file logging:** Utilizes Simulink's native file logging functionality in parallel with the CAN architecture to capture all necessary local variables, serving as the primary data source for post-run testing and debugging analysis.

On figure 5 you can view a typical log analysis study using the Simulink Data Inspector. More information on the log testing and analysis phase is available in the section 7.4.

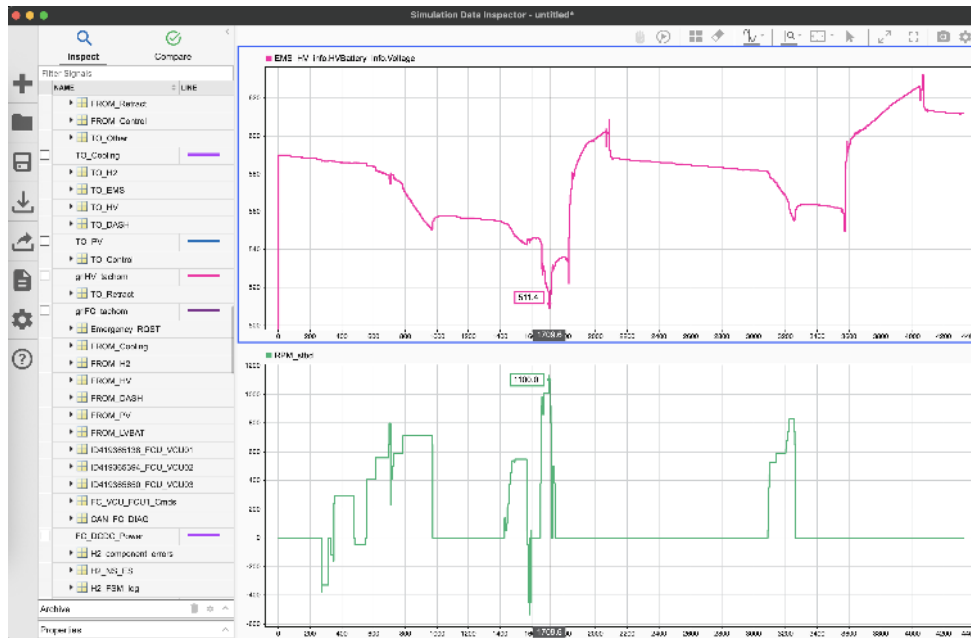


Figure 5: A typical view of the Simulink Data Viewer interface with the HV battery voltage and the starboard RPM command

4 Main FSM

The main FSM is divided into three concurrent child FSMs that execute in parallel. The **Main_Safety_FSM** serves as the primary controller, managing subsystem authorizations while continuously prioritizing pilot safety and hardware protection.

4.1 Truth Tables

The operational authorization logic for each subsystem involves complex interdependencies. To manage this complexity, Stateflow truth tables are utilized. A truth table evaluates a set of logical conditions and executes specific actions based on the outcomes. It can either return a status variable or directly modify system parameters. For detailed implementation mechanics, refer to the MathWorks Stateflow documentation on truth tables [3].

The primary applications of truth tables within this control architecture include:

- Subsystem authorization and mode swaps
- Subsystem and or switches Disabling
- State transition monitoring
- Subsystem state request updates

As an example Figure 6 illustrates the specific truth table implementation utilized to authorize ESC state transitions.

Condition Table				
	DESCRIPTION	CONDITION	D1	D2
1		FROM_HVState_HV == HVState.Off	F	T
2		FROM_DASH_Switch_Status_S_ESC == true	-	F
ACTIONS: SPECIFY A ROW FROM THE ACTION TABLE			3	1

Action Table		
	DESCRIPTION	ACTION
1	allow ESC switch toggle	TO_DASH_Switch_Aliased_S_ESC = true; TO_HVESC_state_req1 = ESCState.Off
2	Turn On ESC	TO_DASH_Switch_Aliased_S_ESC = true; TO_HVESC_state_req1 = ESCState.On
3	Don't allow ESC	TO_DASH_Switch_Aliased_S_ESC = false; TO_HVESC_state_req1 = ESCState.Off

Figure 6: Update_ESC_state truth table. This table is evaluated at every step in the Functional mode to authorize the ESC switch or change its state request depending on the actual HV state

4.2 Safety FSM

This FSM is divided into 6 child states that describe the normal operation of the boat and is visible on Figure 7

- **Init:** Executed at power-up to perform a comprehensive system-wide health check.
- **Startup:** Verifies switches are off and pumps are working.
- **Functional:** The nominal operating mode. The pilot can manually toggle subsystem switches to request state changes depending on the system state.
- **Safety_Shutdown:** Triggered by non-critical faults or warnings to protect system hardware.
- **Emergency:** Instantly isolates critical systems to maximize pilot and hardware protection during high-severity failure events.
- **Maintenance:** Decouples subsystems to allow isolated component power-up and testing independent of the global system logic.

Following any shutdown or emergency sequence, the FSM automatically reverts to the **Init** state to re-evaluate system integrity.

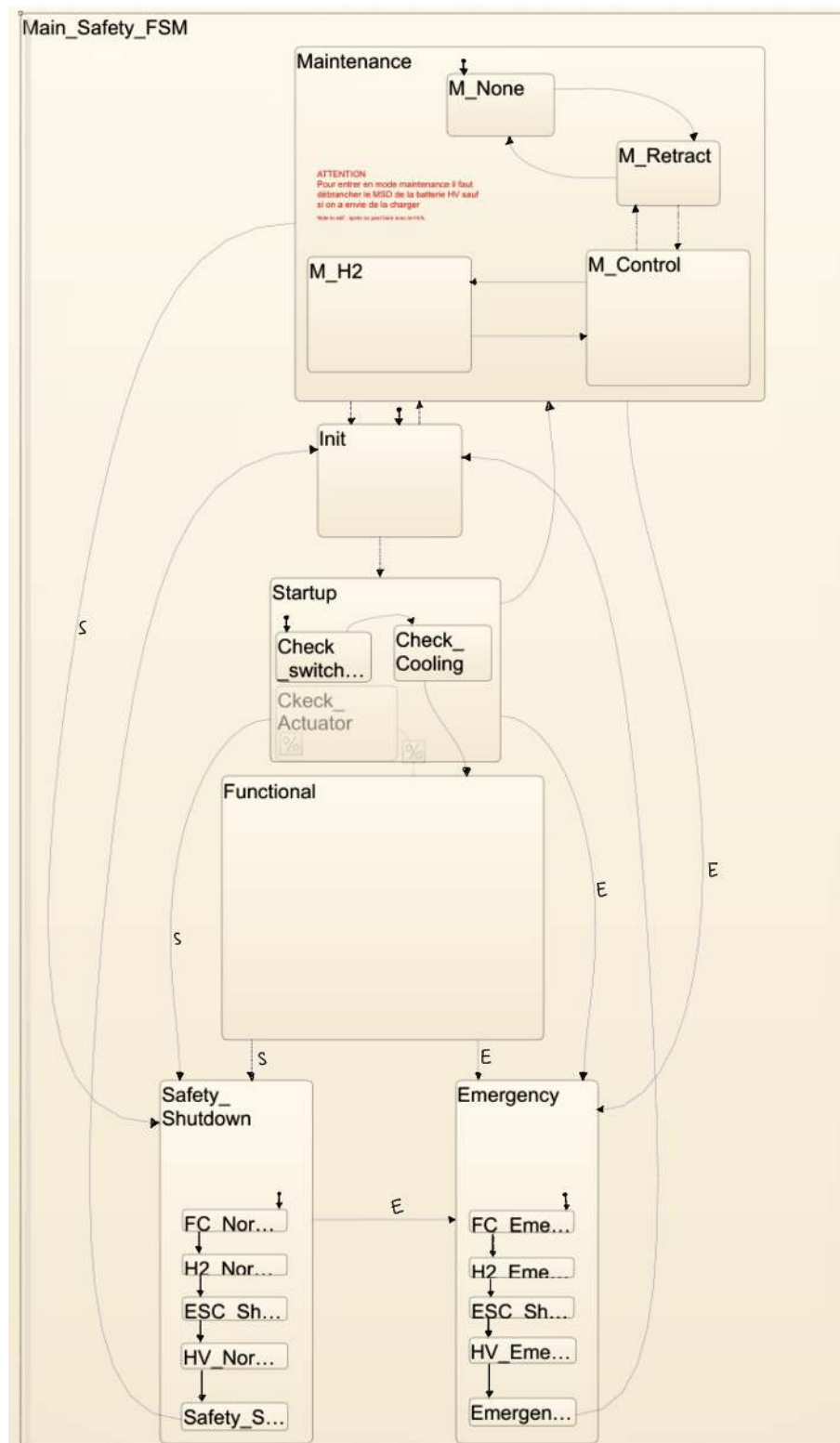


Figure 7: Main safety FSM. This FSM manages all subsystems and their emergencies. S = Safety Shutdown, E = Emergency

4.2.1 Init

During the **Init** phase, the system executes a comprehensive health check across all peripheral subsystems. Every subsystem is initially forced into a disabled state to prevent unexpected hardware behavior.

The physical mode selector and homing button remain active to allow an operator to transition into **Maintenance** mode. A trained operator can utilize XCP bypasses to override an abnormal subsystem or safety interlock (see Section 6.3); this must be exercised with extreme caution.

To successfully transition from **Init** to the **Startup** state, all subsystems must be online, communicating normally, and reporting zero faults or emergency requests.

4.2.2 Startup

The **Startup** phase performs automated sequential pre-operational hardware verification:

1. **Switch Position Verification:** Confirms that all physical switches are in the *OFF* and that the throttle is in the neutral position.
2. **Cooling System Diagnostics:** Briefly cycles and tests both the open-loop and closed-loop cooling pumps to verify hydraulic integrity.

If all checks pass without triggering a system fault, the model automatically transitions to the **Functional** state.

4.2.3 Functional

Upon entering the **Functional** state, the boat is fully operational. Subsystems can be activated at the pilot's request, subject to dynamic authorization based on the current state of the boat.

At the initial transition into this mode, all subsystem activation switches are initially disabled, while control actuators are enabled. Control of each switch is then handed over to its respective subsystem logic; for example, the High-Voltage (HV) switch authorization is governed by the `update_HV` truth table.

1. **Heartbeat Verification:** Confirms that all networked subsystems remain online and communicating.
2. **Subsystem Authorization:** Evaluates permissions for the PV, throttle inputs, and user mode selections.
3. **State Request Routing:** Updates the target state requests for all subsystems.
4. **Hydrogen Solenoid Management:** Links physical switch inputs to the hydrogen system solenoids (the actuation logic remains encapsulated within the dedicated H2 subsystem).
5. **Control Loop Update:** Refreshes active control behaviors and algorithms.
6. **Throttle Command Calculation:** Computes the target RPM command and transmits it to the EMS.
7. **State Change Detection:** Monitors for any active subsystem state transitions.
8. **LV Interlock Check:** Verifies whether conditions permit a safe shutdown of the boat's LV bus.

4.2.4 Safety Shutdown

The **Safety_Shutdown** state handles non-critical faults or warnings to prevent hardware damage and mitigate risk escalation. This procedure manages a controlled shutdown of subsystems and can be initiated from the **Maintenance**, **Startup**, or **Functional** states.

The sequential shutdown procedure executes as follows (see figure 8):

1. **Control Invalidation:** Dashboard switches, XCP communication and low-voltage power-offs are disabled. Motor are commanded to 0 RPM.
2. **FC Normal Shutdown:** Skipped after 65 seconds.
3. **H2 Normal Shutdown:** Skipped after 1 second.
4. **ESC Trun-OFF:** ESCs are dissabled. Skipped after 0.5 second.
5. **HV Normal Shutdown:** The DC-DC converter is turned off, followed by the opening of the HV battery isolation relays.

Once all subsystems confirm a successful shutdown via internal state checks and elapsed timeout delays, the FSM automatically reverts to the **Init** state. Specific fault triggers are encapsulated within each subsystem's internal logic and are omitted here due to frequent updates. For the comprehensive, up-to-date protocol, refer to the active *Safety_Shutdown_Procedure* [4] in the 3.16 Procédures et Sécurité folder.

4.2.5 Emergency Shutdown

The **Emergency** state handles high-severity faults requiring immediate isolation. Accessible from any operational mode (including **Safety_Shutdown**), it executes the exact same sequence as the safety shutdown but utilizes specialized emergency commands and compressed time delays for rapid protection. (see figure 8). The typical time delay to skip an action is 0.4 seconds. Upon validation, the FSM reverts to the **Init** state. Individual subsystem fault triggers and precise timing details are documented in the active *Emergency_Shutdown_Procedure* [5] in the 3.16 Procédures et Sécurité folder.

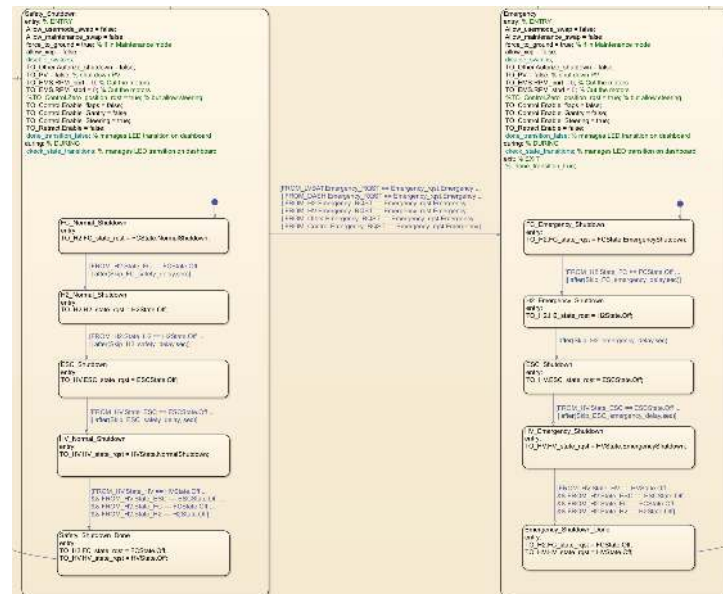


Figure 8: Safety Shutdown and Emergency shutdown procedures. Delays are defined in the `main_FSM_cte.m` file

4.2.6 Maintenance

The Maintenance Mode enables individual subsystem activation. It is used for high-voltage (HV) battery charging, H_2 circuit maintenance, and other subsystem procedures.

Mode activation

- **Entry:** Turn the selector to the 0 position while holding down the homing button.
- **Subsystem Selection:** Turn the selector to the specific maintenance mode while holding down the homing button.

HV Battery Charging

- Charging is always available in the Maintenance state.
- **Connection:** Connect the battery directly to the bidirectional source.
- **Safety Interlock:** When the HV switch is ON, only the HV battery activates. All other HV components remain disabled.

M_retractation (Gantry and Foils)

- Retracts the gantry and sets the pins in the two struts.
- *Note:* Foil retraction can also be performed in Functional Mode.

M_control (Actuator and Algorithm Test)

- Provides the pilot access to control sensors, actuators, steering, and throttle to verify control algorithms.
- **Safety Interlock:** The throttle is disconnected from the torpedos.
- **Simulation:** The spare2 switch simulates either "Harbour" or "Flight" user modes.

M_H2 (Hydrogen Circuit)

- Allows the pilot to manually activate solenoids (e.g., for system purging).
- **Warning:** High-danger mode. Operation is restricted to trained personnel only.

4.3 User FSM

The **User_FSM** manages the pilot's operating mode selection, defining 5 distinct states that dictate subsystem behaviors and authorizations via a physical or software selector.

- **Maintenance (0):** Enables individual subsystem activation. See section 4.2.6
- **Ground (1):** The default power-up state. Torpedoes are disabled. Transition to *Maintenance* mode is permitted only if the subsystem status is **Init** or **Startup**.
- **Harbour (2):** Enables low-speed maneuvering. Torpedoes operate in differential mode at a trimmed speed limit.
- **Archimedean (3):** Standard open-sea mode. Both torpedoes are synchronized, and full power is authorized.
- **Flight (4):** Foil-borne operation. This mode requires the gantry to be mechanically secured (**Pin_locked** active). This is the exclusive state where active flight control algorithms are executed.

Safety Fallbacks

If a system error occurs during operation:

- **Flight** automatically downgrades to **Archimedean**.
- **Archimedean** automatically downgrades to **Harbour**.

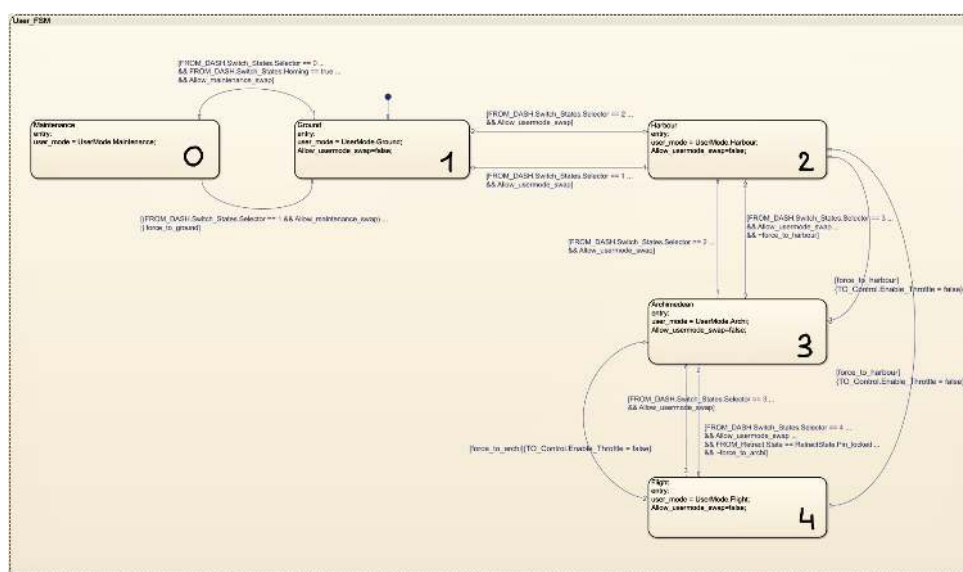


Figure 9: User FSM running in parallel to the main safety fsm

5 Subsystems

This section details the function, critical design points, and technical subtleties of each subsystem. For subsystems developed by external contributors, references to the corresponding technical reports are included along with their defined Inputs/Outputs (I/Os).

To properly review and understand the interface dependencies, open the main Simulink model: `boat_main.slx`.

5.1 Low Voltage Battery - LVBAT

The **LVBAT** subsystem manages the Low-Voltage (LV) battery. Its primary functions are:

1. **Emergency Monitoring:** Decodes incoming battery CAN messages. It triggers an emergency request either upon receiving explicit error flags via CAN, or when telemetry values exceed the safety thresholds defined in the `??` file.
2. **Data Telemetry:** Forwards all relevant battery status parameters to the EMS for global power system configuration.

Critical Implementation Notes

- **Power Routing:** The subsystem operates with no external input dependencies.
- **CAN Multiplexing:** Specific CAN blocks are sampled at 500 Hz to handle signal multiplexing during data decoding.
- **Relay State Interlocking:** The relay states (`Chg_MOS_State` and `Dischg_MOS_State`) trigger an Emergency condition if the battery opens its internal relays while the boat remains active.
- **XCP Override:** `XCP` parameter is implemented to override and bypass the emergency request normally triggered by a loss of battery communication.
- **Undervoltage Protection:** If the total battery voltage drops below the `cell_undervoltage_warning` threshold, the subsystem initiates a safety shutdown to secure the boat before a complete LV power loss occurs.

5.2 Solar Panels - PV

The Solar Panel subsystem structure matches the standard subsystem architecture but is currently implemented as a software stub.

Functional Architecture (Planned)

- **Control Loop:** The EMS transmits configuration settings to determine which solar panels to activate.
- **Authorization:** The main FSM enables the subsystem based on current state permissions.
- **Feedback:** Status data and telemetry are propagated back to both the main FSM and the EMS.

The subsystem currently simulates normal operation (reports online status and zero errors).

5.3 Dashboard - DASH

The dashboard subsystem is one of the most important user interfaces with the ground station. It allows the pilot to activate subsystems when they are authorized and informs the pilot of the behavior of the subsystems through the leds.

The dashboard is divided into two parts: port and starboard. On each parts there are multiple switches and buttons. On the figure 10 you can see the attribution of every switch or button. Some button have multi purposes.

The dashboard subsystem serves as the primary physical user interface between the operator and the system. It enables the pilot to toggle authorized subsystems and provides real-time operational feedback via status LEDs. The hardware interface is physically segmented into port and starboard consoles, each integrating multiple switches and push-buttons. Figure 10 shows the two dashboards. Some switches have multiple purpose.



Figure 10: Picture of the port and starboard dashboard side by side. The big red button is the Emergency Button and the LEMO connector on its right is the LV battery Key. The green sticker on the Mode selector indicates position 1 of the selector

5.3.1 Emergency and Fault Detection

If a switch detects a fault (disconnection or loose connection) or the Emergency button is pressed, the Dashboard sends an `Emergency_rqst` to the main FSM. The Emergency button triggers an Emergency and a switch fault or a dashboard that goes offline triggers a Safety Shtudown. The fault detection is done on the respective dashboard MCU and uses a pull-up/pull-down logic.

5.3.2 Normal Switch FSM

The normal switch FSM just copies the input boolean to the output if the **allow** input is set to true. It is used for buttons or spare switches where with no LED feedback.

5.3.3 Selector FSM

The selector has its own finite state machine. First, set it to position 1. Once in this position, it begins transferring the input to the output if the operation is permitted. If the selector is switched but the operation is not permitted, return it to its previous position.

5.3.4 Switch+LED FSM

This FSM is more complicated, as it handles the switch LED, authorizations, state changes, and fault detection. See figure 11

- If the switch is toggled without authorization, the LED blinks rapidly.
- If toggled with authorization, the LED blinks slowly during the transition, then stays ON once completed. The feedback on the transition comes from the subsystem in question.
- If a transition happens without the pilot action, then the LED blinks quickly so the pilot knows he needs to toggle it.
- If an error occurs, the system resets the switch to OFF via software if permitted; otherwise, it triggers a Safety Shutdown.

5.3.5 Solenoid Switch FSM

This FSM is very similar to the Switch+LED FSM but has no transition delay (just state change).

- If the switch is toggled without authorization, the LED blinks rapidly.
- If toggled with authorization, the LED switches on and the output is set to true.
- If a transition happens without the pilot action, then the LED blinks quickly so the pilot knows he needs to toggle it.
- If an error occurs, the system resets the switch to OFF via software if permitted; otherwise, it triggers an emergency request.

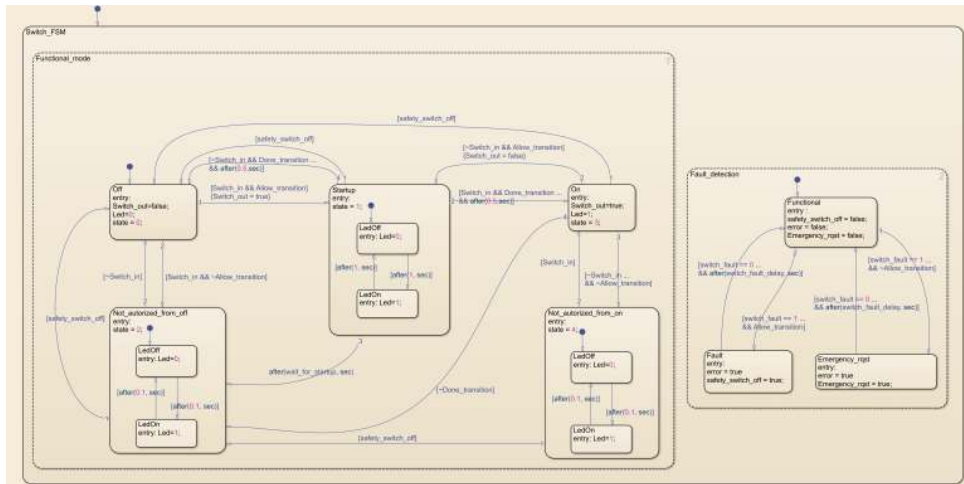


Figure 11: Switch+LED FSM with the functional mode and the fault detection.

Input: Switch_in, Done_transition, Allow_transition

Output: Switch_out, Led, error, Emergency_rqst

5.4 High Voltage - HV

The high-voltage subsystem manages the HV battery, the DC-DC converter, and the 2 ESCs.

HV_logic.slx is a subsystem reference and has been developed by Romain Mauris. You can therefore find more information in his report 2026_Spring_Romain_Mauris_Energy_Management_System [6].

For convenience, all inputs and outputs are described here.

Inputs ———

- **HV_state_rqst**: Requested HV state (On, Off, NormalShutdown, EmergencyShutdown).
- **ESC_state_rqst**: Requested ESC state (On, Off).
- **Maintenance**: Boolean that allows turning on the HV battery only.
- **EMS_Settings**: Information of the EMS to tune the HV settings.
- **IO191_HV_in**: Input from the HVIL sensor.

Outputs ———

- **Online**: Is true if everything is online in the HV subsystem.
- **state_HV**: HV state (On, Off, NormalShutdown, EmergencyShutdown).
- **state_ESC**: ESC state (On, Off).
- **Emergency_RQST**: Emergency request form the HV (None, Safety_Shutdown, Emergency).
- **Errors**: True if there is an error in the subsystem .
- **Info_EMS**: HV information sent to the EMS .
- **Errors_log**: Bus with all errors that can be generated in the HV system.
- **HV_LOG**: State of the FSMs in the system (sent to the raspi to display on the GS) .
- **IO191_HV_out**: output to digital port to command green and red LED (for HVIL).

5.5 Energy Management System - EMS

The Energy Management System takes as input all the information from the energy subsystems as well as the throttle command and sets the speed for the torpedos and the settings of the H2, HV and solar panels. **EMS.slx** is a subsystem reference and has been developed by Romain Mauris. You can therefore find more information in his report 2026_Spring_Romain_Mauris_Energy_Management_System [6].

For convenience, all inputs and outputs are described here.

Inputs ———

- **Main_FSM_info**: Torpedo speed (RPM for port and starboard) and cooling test request (boolean).
- **HV_info**: Battery, ESC and DCDC information.
- **H2_info**: Fuel cell information.
- **LVBAT_info**: LV battery information.
- **PV_info**: Solar panel information.

Outputs ———

- **HV_settings**: DCDC and ESC setpoints.
- **H2_settings**: Desired FC power.
- **PV_settings**: PV enable selection.
- **Cooling_settings**: Pump regime.

5.6 Hydrogen - H2

The Hydrogen subsystem manages the Fuel cell and the H2 circuit (tank and solenoids). It is the only subsystem that can start a shutdown procedure without waiting on the Main_Safety_FSM's command. This is for obvious safety reasons. In other words, if this subsystem encounters an emergency, it starts an Emergency Shutdown and in parallel, it sends to the Main_Safety_FSM an Emergency_rqst to shut down the rest of the boat. This is also the case of a safety shutdown.

H2_logic.slx is a subsystem reference and has been developed by Romain Mauris. You can therefore find more information in his report 2026_Spring_Romain_Mauris_Energy_Management_System [6].

For convenience, all inputs and outputs are described here.

Inputs ———

- **H2_state_rqst**: Requested H2 state (On, Off).
- **FC_state_rqst**: Requested FC state (On, Off, NormalShutdown, EmergencyShutdown).
- **Solenoids_states_rqst**: This is a bus of 3 boolean that corresponds to the requested state of the solenoids (tank, FC_circuit, purge). Their state is true/false.
- **Maintenance**: Boolean that allows maintenance mode.
- **EMS_Settings**: Information of the EMS to tune the HV settings.
- **IO191_H2_in**: Digital pressure sensors.
- **IO397_H2_in**: NTC ATEX sensor and H FAN tachometer.

Outputs ———

- **Online**: Is true if everything is online in the H2 subsystem
- **state_H2**: H2 state (On, Off).
- **state_FC**: FC state (On, Off, NormalShutdown, EmergencyShutdown).
- **Emergency_RQST**: Emergency request form the HV (None, Safety_Shutdown, Emergency).
- **Allow_solenoid_switchs** : Bus of 3 boolean that are true if it is allowed to toggle the solenoid state with the corresponding switch.
- **Solenoid_feedback**: Bus of 3 boolean that gives the state of the solenoids (true = on).
- **Errors**: True if there is an error in the subsystem.
- **Info_EMS**: H2 information sent to the EMS.
- **Errors_log**: Bus with all errors that can be generated in the HV system.
- **H2_LOG**: State of the FSMs in the system (sent to the raspi to display on the GS).
- **IO191_H2_out**: Valve settings (OTV, Purge, Low pressure) and FC wake-up pin.
- **IO397_H2_out**: FAN PWM settings.

5.7 Cooling

The cooling system manages the two maxi puppy pumps and the GRI pumps. The command of the maxi puppy are directly rooted to the IO191, while the command of the GRI pumps goes trough a control logic before being rooted to the IO397.

GRI pumps must be maintained at 0.7 V to remain stopped. At a control voltage of 0 V, the pumps start running at full speed. Due to unusual behavior of the boat (which has not yet been identified), the pump sometimes stops without any control signal being sent to it. To correct this, the logic system monitors the control signal and the pump's actual speed (using the tachometer reading). If the control signal is less than 1 V but the pump is running at high speed, the control signal is set to 2 V for 2 seconds, then reset to 0.7 V. This ensures the pump remains stopped when necessary.

5.8 CPU and Overload Monitoring - Other

This subsystem monitors the Speedgoat activity to ensure system safety. It triggers a Safety Shutdown if the CPU temperature exceeds a threshold (`CPU_max_temp_warning`), and an Emergency Shutdown if it exceeds `CPU_max_temp_emergency`.

To prevent the model from stopping unexpectedly, overloads are permanently authorized via the overload option block. Connecting or disconnecting to the model via Simulink or XCP routinely causes 1 to 5 overflows due to the sudden data-flow workload. Consequently, an Emergency is triggered if an overload occurs while the XCP switch is OFF.

This subsystem is also responsible for holding the LV battery key high to prevent an LV shutdown if not authorized. A function in the Main FSM checks if it is authorized to shut down the boat and sends a boolean to this subsystem.

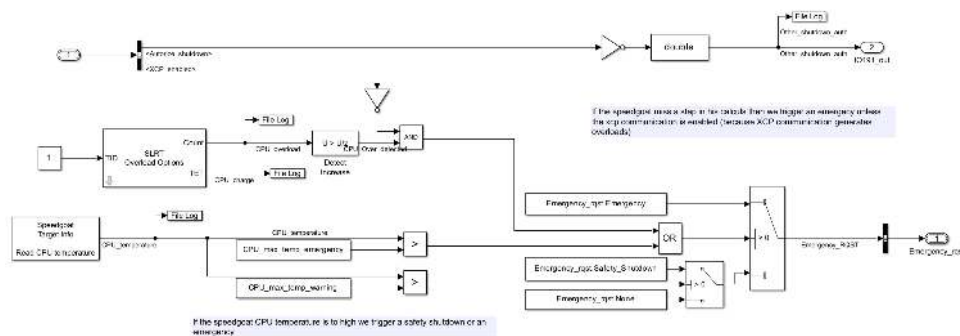


Figure 12: Logic of the overload option bypass.

5.9 Retractation

The Retractions subsystem contains the FSM that controls and synchronizes the foil retraction system.

Retract_logic.slx is a subsystem reference and has been developed by Margaux Lari and Beltran Albert. You can therefore find more information in their semester project report. You can therefore find more information in their report 2026 Spring Lari Albert FoilRetractionSystem [7].

For convenience, all inputs and outputs are described here.

Inputs —

- **Enable:** Boolean to enable the subsystem, linked to the enable retraction switch.
- **Homing_rqst:** Boolean linked to the homing button.
- **Roll1:** Roll switch to set the target height of the port gantry.
- **Roll2:** Roll switch to set the target height of the starboard gantry.
- **Reset_fault:** boolean that allows resetting the system if it has a fault. This is a XCP_short block.

Outputs —

- **Online:** Is true if everything is online in the HV subsystem.
- **State:** State of the retraction (Free, Moving, Pin_locked).
- **Errors:** Is true if there is an error in the subsystem.
- **Errors_log:** Bus with all the errors that can be generated in the HV system.
- **Retract_log:** State of the FSMs in the system (sent to the raspi to display on the GS).

5.10 Control

All blocks within this subsystem handle boat control and are managed by the `control_logic` subsystem reference. On the input side, the "Control Sensors" block contains the CAN drivers for the IMU and ultrasound sensors, transmitting telemetry directly to the control logic at dedicated sampling frequencies alongside steering and throttle commands.

The `control_logic` block (subsystem reference) processes these inputs to output position commands for the six physical actuators: two gantries, two flaps, the rudder, and the stabilizer. It executes the active flight control algorithms during **Flight** mode and handles the kinematic mapping from the steering wheel to the actuators. This subsystem reference was developed by the Control Team; for detailed theoretical and algorithmic implementation, refer to their specific semester project report. Alessandro Lupi worked on the actuation system, his report is available in the this folder [8].

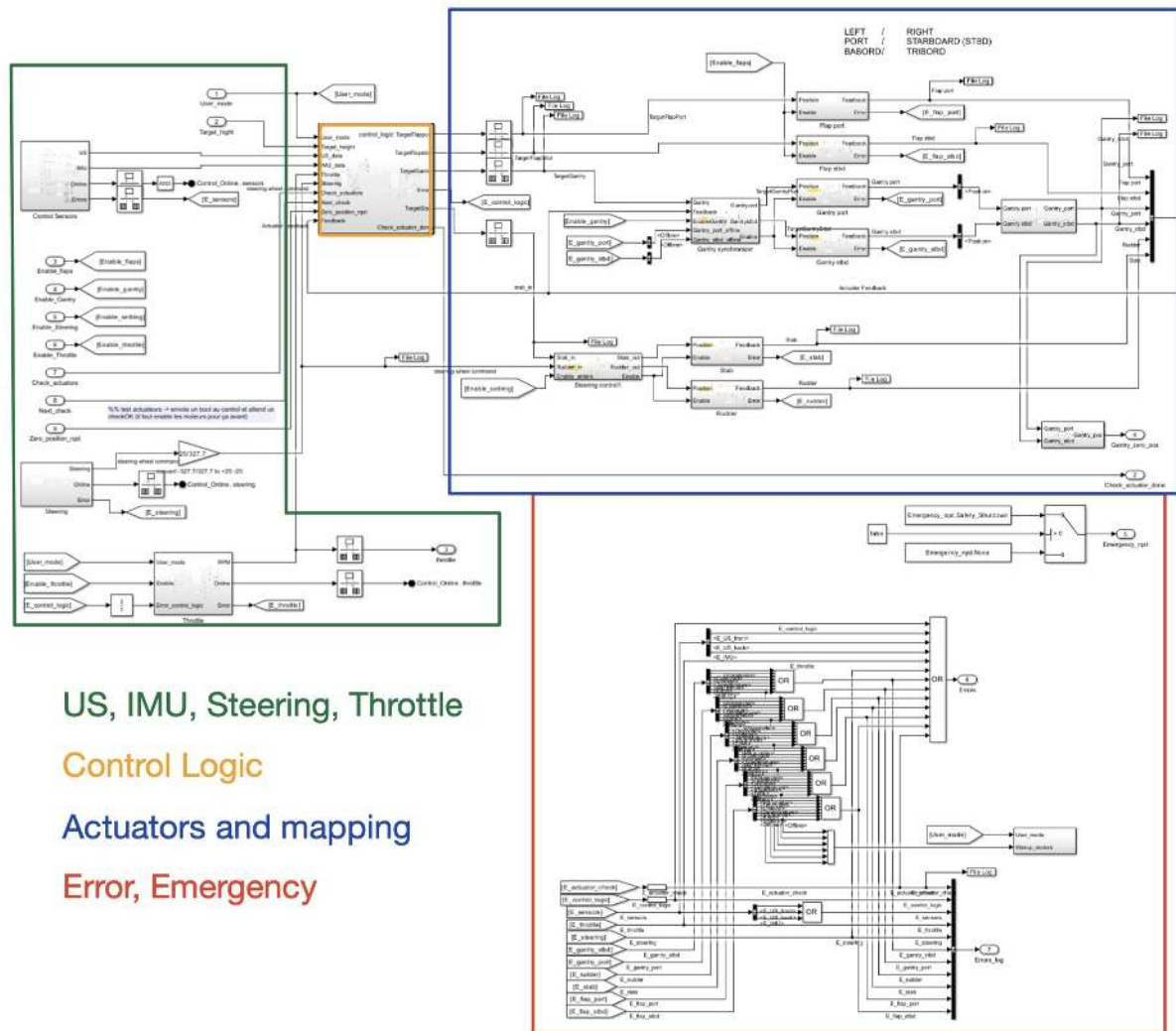


Figure 13: Control subsystem with its different sections. The control logic is a reference subsystem

5.10.1 Throttle

The command received by CAN from the throttle goes through an FSM that changes the behavior of the torpedo depending on the user mode. It also makes it possible to disable the torpedo by sending a zero RPM command.

If in **Harbour** mode the throttle is in differential and trimmed mode. The pilot is free to activate only one torpedo or one in each direction. The first gear is motor idle (the motor turns but does not transmit power). The second gear is trimmed to a certain value. It is possible to set the motor backward to maneuver in the harbour.

If in **Archi** or **Flight** mode the two torpedoes are still differential, but it is now possible to use full power, up to 3'050 RPM in positive command and up to 1'050 in negative command.

5.10.2 Steering Control

This subsystem synchronizes the rudder and the stab command with a lookup table. This is necessary because of the coupling of the two actuators through the foil holder.

5.10.3 Gantry synchroniser

The gantry synchronizer is here to synchronize the two gantry motors. It gets a target angle as input and gives the command to the two actuators. It uses a feedback loop with a simple P controller to avoid slowing down the control.

5.10.4 Subtleties

The actuators are controlled via an abstraction library that maps position commands to specific hardware node IDs and automatically executes a wake-up sequence if a node goes offline.

To mitigate issues with actuator disconnection a dedicated subsystem forces a global actuator wake-up sequence when an actuator goes offline and when the system is in the **GROUND** state. Currently, the control subsystem does not trigger emergency requests, but the underlying detection logic is implemented to support future safety features.

6 Desing Choices

6.1 Code Modularity

The software architecture utilizes a modular design to simplify system modifications and scaling. New subsystems can be integrated into the existing framework by defining a new signal bus or extending an existing bus structure. Several features are currently implemented but commented out due to project delays or immediate operational requirements; these can be reactivated as required for future developments.

6.2 Frequencies

The base execution frequency of the main Simulink model is set to 500 Hz. This high rate is strictly dictated by the CAN interface requirements of the low-voltage battery, which utilizes signal multiplexing and demands rapid polling to ensure complete data frame capture.

Subsystem Sampling Rates

- **Main Core Processes:** Operate at 100 Hz. While a lower rate (e.g., 20 Hz) is typically sufficient for human-in-the-loop systems, 100 Hz was selected to provide a conservative baseline for initial testing. Current logs shows total Speedgoat CPU utilization remains below 15%.
- **Dashboard (DASH):** Sampled at 10 Hz that is enough for human interaction.
- **Control Subsystem:** Utilizes dedicated frequencies configured to match their algorithmic filter requirements.
- **Electronic Speed Controllers (ESCs):** Transmit messages at varied, prioritized frequencies to optimize bandwidth and minimize overall CAN bus load.

Multi-Rate Data Integrity

To manage data exchange across different execution domains and maintain deterministic behavior:

- **Rate Transition Blocks** handle data serialization between subsystems operating at different sampling rates.
- **Unit Delay Blocks** are strategically inserted to break algebraic loops created by the main FSM feedback paths.

All global frequency constants are defined in the initialization script `frequency_cte.m`. This file is subject to frequent updates during optimization; refer directly to the repository for the active configuration.

6.3 XCP

XCP is the communication protocol used to change parameters in the model during run time. It uses the GS on the raspi to setup an A2L file (that is proper to the model) and to communicate with the speedgoat.

As it is very dangerous to change variables of the model while running, here are some safeguards.

- Only trained persons should use the XCP when the model is running
- Every variable that we explicitly know we want to modify with XCP are defined in the XCP_Variables.m file and are denoted with XCP at the beginning.
- XCP usage is only allowed in Init, Startup, Maintenance and when it is also authorized to do a complete shutdown of the boat in Functional mode.

XCP needs to be handled with care because it is source of a lot of compilation and Simulink errors. Two XCP_Short blocs are available in the project library. The normal XCP_Short allows overriding a Boolean or an Enum by using the equivalent double input. Do NOT use something else as double as XCP input!

The second XCP library block allows overriding a double signal. On the GS you only can read double and boolean.

6.3.1 A2L File Generation

As mentioned above, to use XCP communication, it is necessary to create an A2L file of the project and upload it on the GS under the Dashboard tab.

1. Compile the project with the C++ compiler or click "Run On Target" to upload the model to the Speedgoat.
2. C Code > Share > Generate Calibration Files
3. select the folder path where you want the A2L to be saved
4. Browse for a symbol file into the folder boat_main_sg_rtw that has exactly the name: "boat_main" without extensions.
5. Press the generate button

To upload the A2L file into the GS you would use the following code: goodjob (used last time June 2026, possibly not the same now).

6.4 Simulink Specifics

- **Atomic Subsystems:** Forces Simulink to compile code blocks as single, non-reordered execution entities to ensure determinism.
- **Simulink Solver Type:** Configured as a **Discrete, fixed-step solver** 500 Hz to match the highest system frequency.
- **Enumerations:** Defined explicitly to comply with C++ code generation requirements (see templates in /data_type/enum).
- **Bus Definitions:** Strictly typed interfaces are enforced between subsystems and the main FSM to prevent structural compilation errors.

7 Tests

The testing of the Simulink was done in different phases depending on the project's progress. Because of the short timelines of this semester it was not easy to set priorities and extensively test everything. The testing phases were therefore carried out as quickly as possible to allow the project to move forward. Further testing is needed to ensure that the code works properly even in edge cases.

7.1 Main FSM Testing

The main FSM was tested standalone in a separate file. Inputs were controlled via constants and switches, and outputs were monitored using displays.

The test procedure was as follows:

1. Verified all normal transitions between the main states (*Init*, *Startup*, *Functional*, *Safety Shutdown*, *Emergency*, *Maintenance*) in the security FSM.
2. Verified that the emergency procedure behaves as specified.
3. Verified that all transitions in the user FSM were reachable and their conditions were correct.
4. Verified all conditions in every *Functional* mode truth table by applying targeted test inputs and validating the outputs against expected results.

7.2 Subsystem Testing

Each subsystem was tested separately before integration into the `boat_main` Simulink model. Testing was conducted by the responsible person either during this semester or the previous one.

The systems were validated using two primary approaches:

- **Hardware Testing:** Every hardware component was verified prior to installation on the boat (e.g., the LV testbench and the motor testbench). This phase began last semester and concluded at the start of the current semester.
- **Software Logic Testing:**
 - *Low-complexity logic:* Subsystems were tested directly on the boat using a dedicated test Simulink model. In some cases, peripheral subsystems (e.g., the dashboard) were included to validate end-to-end functionality.
 - *High-complexity logic:* Subsystems were first validated via simulation before being deployed and tested on the boat.

7.3 System As-a-whole Testing

System testing was executed incrementally. At each step, the emergency button was verified to ensure the emergency shutdown procedure functions correctly if a test fails.

The validation steps were performed in the following sequence:

1. **Maintenance Validation:** Verified emergency transitions, HV battery charging, foil retraction, and the H2 maintenance mode.
2. **State Transition Validation:** Verified the transition from *Init* to *Startup* to ensure all subsystems correctly transmit their error and online statuses. The transition from *Startup* to *Functional* was then validated.
3. **Subsystem Integration Testing:** Verified the simultaneous operation and fusion of all subsystems within the *Functional* mode.

7.4 Logs Analysis

The Simulink Data Inspector was a critical tool during these tests. Combined with a structured logging strategy (file logs), it enabled model behavior tracking and post-test data processing. For example, Figure 14 illustrates a standard log analysis used to diagnose a Safety Shutdown triggered by an ESC.

All logs are collected in the LOGS folder, where you will find the previous records from the initial testing on the lake.

Important

Simulink compilation is time-consuming. To maximize efficiency in future development, ensure all required signals are configured for logging before initiating a test. It is more efficient to spend time configuring proper log files initially than to complete a test with insufficient logs and be forced to re-run it.

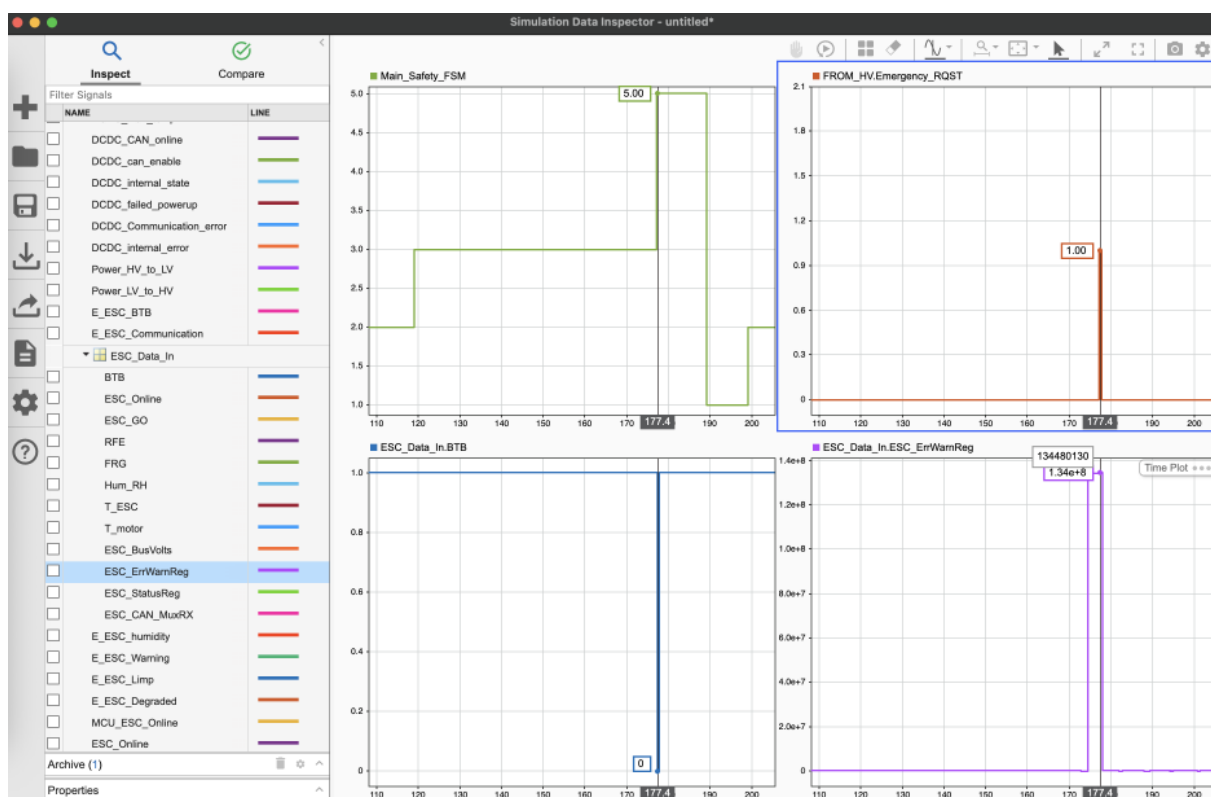


Figure 14: Analysis example of a safety shutdown caused by the starboard ESC. The `Main_Safety_FSM` first transitions to `Safety_Shutdown`. The `FROM_HV.Emergency_RQST` log isolates the issue to the HV subsystem, while the `BTB` signal confirms a starboard ESC error. The `ESC_Data_In.ESC_ErrWarnReg` register identifies the exact error via the bitmap definition found in the ESC Simulink block. Note: The system contains two ESCs; the second register in the list corresponds to the starboard ESC.

8 Summary of the results

This project can be summarized by different achievements of the whole team during this semester. The different technical achievements demonstrate that the code works properly.

1. Rudder actuation with the steering wheel
2. Dashboard linking to respective subsystems
3. High voltage to low voltage power transmission
4. Torpedo activation with the throttle lever
5. High voltage battery charging with the Fuel cell
6. Validated emergency procedure triggered by different sources (subsystem or emergency button)

8.1 Solutions and important informations

List of solutions and important values		
Title	Description	Reference
Project Gitlab	The project code is available on the SSB Gitlab under the main branch	Boat_repository
Matlab Environment Installation	All useful information to set up the correct environment	Simulink installation (04.2026 version)
Procedures	Procedures to turn ON the boat, detailed emergency procedures and maintenance procedure are available on the drive	Start-up and Shutdown Procedures
Simulink Structure	Hierarchical structure with abstraction levels for a “safety first” approach	see chapter 3
Linked Semester Projects	Projects on the EMS, HV, H2, Retractation and Control	2026 spring semester projects

Table 1: Important links of the project

8.2 Workflow

The project was executed in three sequential phases:

- **Phase 1:** Core software architecture, subsystem design, and unit testing.
- **Phase 2:** Subsystem integration and debugging.
- **Phase 3:** System-wide validation and testing.

An Agile-inspired methodology was adopted to enable rapid iteration, continuous debugging, and the flexible integration of new requirements throughout the code development. A diagram of the workflow is shown in figure 15.

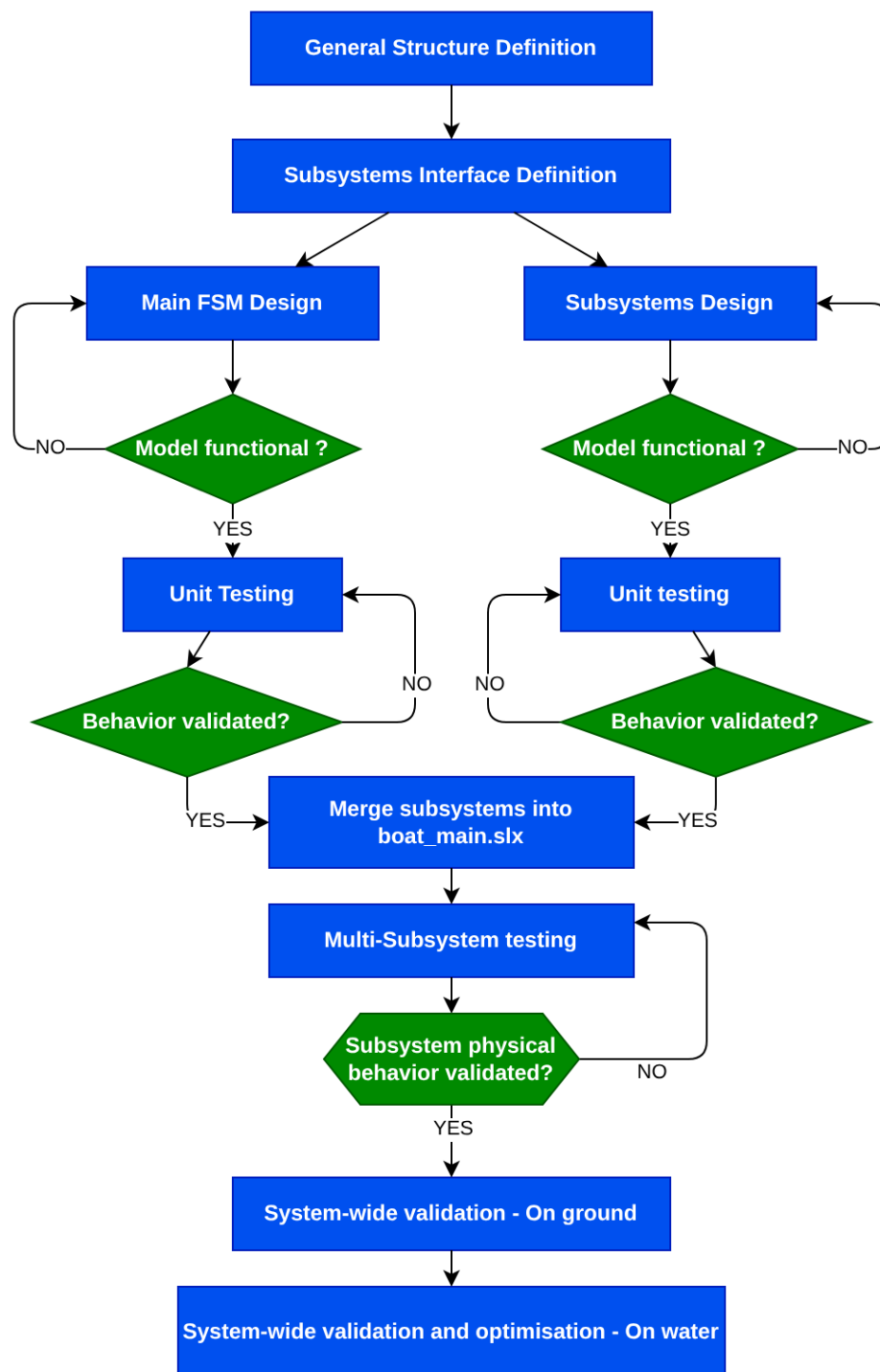


Figure 15: Flow chart of the work done this semester

9 Challenges

This section presents some technical challenges encountered during development and their corresponding solutions.

9.1 Finite State Machine (FSM) Testing

Testing the main FSM was challenging due to high interdependence between components.

- **Solution:** Test the FSM isolation by applying deterministic constant inputs and validating the outputs via system displays.

9.2 Compilation and Simulation Errors

Simulink model compilation differs significantly from the C++ code generation required for the Speedgoat target.

- **C++ Code Generation:** Regularly compile the model in C++ to verify compatibility, particularly when introducing new data types such as enums. Note that C++ compilation can take up to 30 minutes.
- **Simulink Model Updates:** Update the Simulink model frequently. Accumulating too many errors causes Simulink to generate ambiguous or misleading error messages, which often requires reverting to a clean state to debug.
- **Frequency Errors:** Encountering a frequency-related error typically indicates that the model structure is correct, and only timing or sample rates need adjustment.

9.3 DBC Managment

Simulink is picky with uploading a dbc file through the Database Manager because for the structure of the project all the DBC and corresponding libraries are in a subfolder. For convinience here is a "working" process to upload a DBC.

1. It is important that your DBC has a totally new name (not used before). Otherwise simulink remembers it and does some weird stuff
2. Put the .dbc file in the /dbc folder
3. Add a DB CANREAD block and open the Database Manager
4. Press on Add, select the dbc and press enter
5. Change the path of the dbc by clicking on the name in the list. It should begin with dbc/
6. Select the good line and press the button Generate
7. Copy and paste the generated lib file from the main folder to the /dbc folder
8. Delete the lib in the main folder

9.4 Teamwork and Simulink Merges

Collaborating on Simulink is difficult because there is no suitable solution available yet. Merging an .m file works well because it is simply code, but merging a Simulink file (.slx) must be done manually. Subsystem references are designed to facilitate merging, as they are separate files from boat_main.slx. It is therefore possible to work on a subsystem reference in parallel with someone working on 'boat_main'. When merging, the old subsystem reference file can simply be replaced with the new one. For this operation, Simulink must be closed.

For larger merges, it is recommended to leave Simulink open and copy and paste the new code into the desired location in Simulink. This can lead to dependency or DBC file issues; therefore, it is very important to visually check the code afterward and compile it to ensure that everything went smoothly.

10 Conclusion

This project successfully delivered the central software foundation required for the commissioning of the Lobst'air, meeting the primary objective of producing a functional boat before the end of the semester.

The architecture, detailed in Chapter 3, follows a modular and safety-first design philosophy. All subsystems — from high voltage and hydrogen management to foil retraction and motor control — are unified under a single real-time Simulink model executing on the Speedgoat. At the heart of this architecture, the Main FSM, described in Chapter 4, ensures continuous monitoring of every component and provides two distinct shutdown procedures capable of isolating the system within fractions of a second in the event of a failure. This safety-driven approach proved essential during on-water testing, where the emergency procedure was successfully triggered and validated from multiple independent sources.

The integration and testing phases, conducted across three progressive phases from unit testing to full system validation, confirmed that all key functional objectives were met: steering and rudder actuation, torpedo propulsion control, high-voltage and fuel cell energy management, and dashboard-to-subsystem interfacing all operated as intended. The boat reached 19 knots at max speed, demonstrating real-world operational readiness.

10.1 Next Steps

Here is a non-exhaustive list of areas that need improvement or further development. Some of these areas may overlap with other semester-long projects.

- Reliability improvement of the main FSM by edge case testing.
- Optimize the main FSM to be less restrictive on safety shutdown and emergency shutdown. For example if we have a H2 warning maybe it is better to just turn off the H2 system without total shutdown.
- Automate the maintenance procedures of the H2
- Add an automated self-diagnostic during the "Init" and "Startup" modes to better pinpoint where a problem is if there is one.
- Improve the cooling control
- Optimize the model on the frequency side and maybe on memory usage (if necessary because the speedgoat is really powerful)
- Implement the solar panel management
- Optimize the EMS to better manage energy flow and consumption
- Implement a real and reliable logging strategy for further performance tests

References

- [1] SSB, “Boat repository on the ssb gitlab,” <https://gitlab.epfl.ch/swiss-solar-boat1/ref/speedgoat/boat>.
- [2] Mathworks, “Simulink documentation,” <https://ch.mathworks.com/help/simulink/index.html>, 2026.
- [3] —, “Simulink documentation,” <https://ch.mathworks.com/help/stateflow/ug/what-is-a-truth-table.html>, 2026.
- [4] T. Kobler, “Safety_shutdown_procedure,” https://drive.google.com/drive/u/0/folders/1Nk9NGUDixEWz_7Q-3_R0PN0DcJvRDI7a, 2026, mAJ le 30 juin 2026.
- [5] —, “Emergency_shutdown_procedure,” https://drive.google.com/drive/u/0/folders/1Nk9NGUDixEWz_7Q-3_R0PN0DcJvRDI7a, 2026, mAJ le 30 juin 2026.
- [6] R. Mauris, “Energy managment system,” <https://drive.google.com/drive/u/0/folders/1Q9PB9zBQyqQIB52HWgrZJecXqbEoVA4A>, Spring 2026.
- [7] M. Lari and B. Albert, “Foil retraction system,” <https://drive.google.com/file/d/1L9tYym-5WvlqojSk4-OQYoIiee9QT3gH/view>, Spring 2026.
- [8] A. Lupi, “Control integration,” <https://drive.google.com/drive/u/0/folders/1Q9PB9zBQyqQIB52HWgrZJecXqbEoVA4A>, Spring 2026.